

AWS GenAI Developer — Domains 4 & 5 (Hinglish)

Operational Efficiency (4.1–4.3) aur Testing, Validation & Troubleshooting (5.1–5.2)

Domain 3 tak humne dekha ki system **kaam kare aur safe rahe**. Domain 4 dekhta hai ki wo **efficiently chale**, aur Domain 5 ki **wo sach mein kaam kar raha hai, yeh sabit kaise karein**.

4.1 -> COST (kam paise)

4.2 -> SPEED (kam latency, zyada throughput)

4.3 -> MONITORING (GenAI aise fail hota hai jo normal monitoring nahi pakadti)

5.1 -> EVALUATION (kitna achha hai? proactive)

5.2 -> TROUBLESHOOTING (kharab kyun hai? reactive)

Task 4.1 — Cost Optimization

Bedrock cost optimization ka order

1. Select appropriate model

2. Need customization? -> explore options in correct order

3. Prompt engineering aur management

4. Design efficient agents

5. Select correct consumption option

Model selection PEHLE hai, consumption option (on-demand vs provisioned) AAKHIR mein.

Kyun: sasta model chun lena provisioned throughput optimize karne se **bahut zyada** bachata hai. **Bade lever pehle**.

Token efficiency

Bedrock mein aap **tokens ke paise** dete ho — input aur output dono. **Output aksar mehenga hota hai**.

Matlab: har extra shabd prompt mein = paisa. Har lamba jawaab = paisa.

Technique	Kya
Token counting	API call se pehle — cost predict, limit check
Recursive summarization	50-page document → 2 page. Tokens 25x kam
Relevant info pehle	Truncation-safe, aur models shuruaat par zyada dhyan dete hain
Context pruning	RAG ne 10 chunks diye, 3 hi kaam ke — baaki hata do

Summarization mein bhi model call lagta hai. Tabhi worth hai jab document **baar-baar use ho** — ek baar summarize karo, kai baar use karo.

CloudWatch se track karo ki **kaunsi query type** sabse zyada tokens kha rahi hai. Wahi optimize karo.

Model selection — sabse bada lever

1. QUALITY THRESHOLD pehle define karo

2. Bedrock Model Evaluation se test, cost track karte hue

3. Sabse SASTA model jo threshold MEET kare

Target sabse achha model nahi hai — **sabse sasta jo kaafi achha hai**.

Threshold pehle define karna zaroori hai. Warna aap hamesha sabse strong model chun loge.

Price-to-performance:

Model A: 94% accuracy, \$0.015 -> \$0.016 per accuracy point

Model B: 89% accuracy, \$0.003 -> \$0.003 per accuracy point

Model B 5x behtar value hai — par kya 89% aapke use case ke liye kaafi hai? Threshold batata hai.

	Mechanism	Cost
Tiered routing	Complexity pehle assess, phir sahi model	Ek call — sasta
Cascading	Chhota try, confidence kam toh escalate	Do call jab escalate ho

Cost attribution

"GenAI par \$50,000 lag rahe hain" — kaunsi team? kaunsa feature?

Bina attribution ke aap optimize nahi kar sakte, kyunki **pata hi nahi ki kharcha kahan ho raha hai**. Cost Explorer + allocation tags.

Batch ka example:

```
GALAT: user product page kholta hai -> description generate karo
SAHI:  raat ko batch job -> saari descriptions pehle se banao
```

Batch ~**50% sasta** hai, aur same description baar-baar generate nahi hoti.

Caching — sabse seedha saving

Sabse sasta model call wo hai jo hota hi nahi.

Semantic caching — exact match kyun bekaar hai:

```
"mera balance kya hai?"
"balance batao"
"kitna paisa hai account mein?"
```

Teeno ka jawaab same. Par exact match teeno ko alag samajhta hai
-> 3 model calls.

```
query -> embedding -> vector store mein similar dhoondho
      -> similarity 85-95%? -> CACHED jawaab do
```

Threshold ka trade-off: bahut kam (70%) → galat jawaab mil sakta hai. Bahut zyada (99%) → lagbhag exact match, faayda kam.

Yeh embeddings ka **teesra use** hai — 1.5 mein search, 3.1 mein verification, ab caching.

Prompt caching — alag cheez hai:

```
system instructions  <- HAR BAAR same
few-shot examples   <- HAR BAAR same
user question        <- alag
```

Static hissa model side par cache -> ~90% discount

Static hissa prompt ke SHURU mein hona chahiye — caching prefix par kaam karti hai.

```
SAHI:  [static instructions][examples][user query]
GALAT: [user query][static instructions]
```

Cache hit rate monitor karo. 5% hit rate matlab cache **paisa bacha nahi rahi, kharch kar rahi hai** (storage + lookup cost).

Queue ka cost angle

Bina queue: PEAK capacity chahiye (baaki time khaali baithti hai)
Queue ke saath: AVERAGE capacity kaafi, peak wait karta hai

Matlab kam infrastructure, kam paisa. Trade-off: latency badhti hai.

Task 4.2 — Performance Optimization

Do alag cheezein

Actual latency	-> sach mein kitna time laga
Perceived latency	-> USER ko kitna laga

Streaming perceived latency badalti hai, actual nahi.

Bina streaming: 8 second spinner, phir poora jawaab

Streaming: 0.5 second mein pehla shabd

Total time wahi 8 second hai. **Par experience 16x behtar.**

Exam signal: "improve user experience" → streaming. **"Reduce processing time" → streaming se kuch nahi hoga.**

Pre-computation: model call 3 second, DynamoDB lookup **10 millisecond**. Predictable queries pehle se bana lo.

Latency-optimized Bedrock models: same model, tez inference — par **quality ka thoda trade-off**.

Retrieval performance — vector index algorithms

Algorithm	Kaise	Trade-off
HNSW	Graph-based, layers mein	Tez + high recall, memory zyada
IVF	Clusters mein baant kar	Memory kam, bade datasets ke liye behtar
LSH	Hashing	Tez, kam accurate
PQ	Compression	Memory bahut kam, accuracy thodi kam

HNSW parameter	Kis par asar
m	Recall, memory , build time
ef_construction	Build time (search speed par nahi)
ef_search	Query time aur recall

Koi universal best nahi hai — apne data par benchmark karo. Agar sawaal mein memory pressure at scale ho, toh IVF ya PQ.

Custom scoring — similarity kaafi nahi

Final score = relevance x w1 + RECENCY x w2 + authority x w3
--

2010 ka perfectly relevant document purana ho sakta hai. Task 1.4 mein precedential value se rerank tha — yeh wahi idea, index level par.

Latency vs throughput

Latency	-> EK request kitni jaldi
Throughput	-> kitne requests per second

Dono aksar ulte hain: batching throughput badhata hai par per-request latency badhata hai.

Unlimited parallelism throttling laata hai — 429 errors, retries, aur ulta slow. Step Functions Map state mein `MaxConcurrency` set karo.

top_k vs top_p

top_k = 50	-> HAMESHA 50 candidates
top_p = 0.9	-> jitne tokens mila kar 90% probability banaayein

top_p adaptive hai: model confident hai toh kam candidates, confused hai toh zyada. **Isliye aksar behtar hai.**

Aur wahi purani warning: **temperature ya top_p — ek chuno, dono nahi.**

Resource allocation

GenAI workloads ki dikkat: significant compute aur memory, variable latency, specific throughput quotas — **conventional patterns follow nahi karte, isliye traditional FinOps kaafi nahi.**

Service	Kya
Inference Recommender	Automated load testing → optimal instance type recommend . Guess nahi, measure.
EC2 Inferentia	Purpose-built inference chip (Trainium training ke liye)

Token lengths monitor karna GenAI-specific hai. Normal apps mein request size fix hoti hai; yahan wo **utilization ka main driver** hai.

Predictive scaling kyun: reactive scaling **der se aati hai** — jab tak scale ho, spike nikal chuki hoti hai. Market open/close jaise known patterns ke liye pehle se tayaar raho.

Task 4.3 — Monitoring

Central idea — sabse zaroori baat

GenAI systems aise fail hote hain jo traditional ML monitoring pakad hi nahi sakti.

Error rate 0%
Latency normal
System "healthy" dikh raha hai
-> aur jawaab JH00TH hai

Failure mode	Kya
Hallucination	Plausible lagta hai, par fabricated
Inconsistent reasoning	Same sawaal, alag jawaab
Context sensitivity	Prompt mein chhota badlav → output bilkul alag

Teesra testing ko mushkil banata hai — aap sab variations test nahi kar sakte.

End-to-end prompt tracing

Ek prompt ka poora safar:
knowledge base retrieval
tools jo call huye
model invocation
response

Ab pata chal sakta hai ki problem kahan thi. Task 2.5 mein maine kaha tha — retrieval results log karna zaroori hai warna blame galat jagah jaayega. Prompt tracing wahi karta hai, built-in.

Business impact metrics

Technical -> latency, error rate, tokens
Business -> resolved queries, ESCALATIONS, satisfaction

Latency 2.0s se 1.8s ho gayi — **kisko farq padta hai?**
"Escalation rate 20% kam hui" — **wo paisa hai.**

CloudWatch anomaly detection

Baseline **khud seekhta hai** historical data se — aapko threshold guess nahi karna padta. **Aur seasonality samajhta hai** — Monday morning spike normal hai, alert nahi karega.

RUM — client-side truth

Server-side: "response 2 second mein bheja"
RUM: "user ne 5 second baad dekha"

Beech mein network, rendering, client processing hai. **GenAI mein khaas** — streaming mein user experience server metrics se bilkul alag hoti hai.

Correlation — sabse chalak insight

Agar user jawaab ke **turant baad human agent maanga** — **wo jawaab kaam ka nahi tha**, chahe technically theek ho. Yeh sirf correlation se dikhta hai.

Tool monitoring

Tool **"success"** aur tool **"effective"** alag cheezein hain. API ne 200 diya = success. Agent ka kaam hua = effective.

Anomaly: ek tool jo roz 100 baar call hota tha, aaj 5000 baar — **agent loop mein hai?**

Teen troubleshooting techniques — sabse important hissa

1. Golden datasets — hallucination pakadta hai

Verified facts ka curated dataset
-> SageMaker Processing jobs periodically test karein
-> CloudWatch trigger: model update par, ya scheduled
-> JUDGE model responses evaluate kare
-> HALLUCINATION SCORE nikalo

Ab hallucination ek number ban gaya jo track aur alarm ho sakta hai.

2. Output diffing — inconsistency pakadta hai

"How should I save for retirement?"
"What's the best retirement strategy for me?"

Matlab same hai. Jawaab bhi same hona chahiye.
Alag hai -> context sensitivity problem

Method	Compare karta hai
Variation testing	Ek waqt par, alag phrasings
Temporal consistency	Ek phrasing, alag waqt (drift)

Dono chahiye — pehla context sensitivity pakadta hai, doosra drift.

3. Reasoning path tracing — logical error pakadta hai

Bina reasoning ke aapko sirf pata hai ki jawaab galat hai. Reasoning ke saath pata hai ki KAUNSE STEP par galat hua.

Circular reasoning: "A sach hai kyunki B, aur B sach hai kyunki A." **Fluent lagta hai, par khokhla hai.**

Symbolic checks — financial mein math actually verify karo, model par bharosa mat karo.

Automated triage — kyun zaroori

Failure type	Ilaaj
Hallucination	Grounding badhao, RAG improve karo
Reasoning error	CoT prompt theek karo
Inconsistency	Temperature kam karo, prompt tighten karo

Teeno ka ilaaj alag hai. Galat diagnosis = galat fix.

Aur **koi ek technique teeno nahi pakad sakti.**

Continuous learning loop: har naya failure **golden dataset ka hissa** ban jaata hai — dataset khud badhta rehta hai.

Task 5.1 — Evaluation Systems

Core idea

Traditional ML	-> accuracy, precision, recall (EK sahi jawaab)
GenAI	-> relevance, factual accuracy, consistency, fluency (KAI sahi jawaab)

Chaaron alag hain. Jawaab fluent ho sakta hai par galat. Accurate ho sakta hai par irrelevant.

Financial version mein do domain-specific dimensions: compliance aur **tone** — yeh generic metrics mein aate hi nahi.

LLM-as-Judge — do models

Generator model	-> jawaab banata hai
Evaluator model	-> jawaab ko score karta hai

Do ALAG models hone chahiye. Agar same model apna kaam judge karega toh bias aayega.

Output mein **explanations** bhi aane chahiye, sirf score nahi — "3/5" se aap kuch fix nahi kar sakte.

Bring Your Own Inference — naya concept

Bedrock Evaluations **environment-agnostic** hai — model **kahin bhi** hosted ho (Bedrock, SageMaker, apna server, doosra cloud), use evaluate kar sakta hai.

Fayda: aap models compare kar sakte ho chahe wo alag jagah chalein.

A/B testing vs Canary

Technique	Maqsad
A/B testing	Compare — kaunsa behtar hai
Canary	Risk kam karo — safely roll out. Compare karne ke liye nahi.

Feedback — do audiences

Source	Kya batata hai
Customers	Kya <i>helpful</i> tha?
Domain experts	Kya <i>correct</i> tha? (SageMaker Ground Truth)

Customer ko aksar pata hi nahi chalta ki jawaab galat hai — wo confident lagta hai. **Expert review hi factual aur regulatory errors pakadta hai.**

Quality gates — sabse important QA mechanism

Deployment pipeline mein gate
-> accuracy ya compliance threshold se neeche?
-> RELEASE BLOCK

Kharab model production tak pahunch hi nahi sakta — aur yeh faisla us waqt ke insaani judgment par nahi chhoda.

Evaluation datasets bhi versioned hone chahiye. Agar test set badal gaya, toh v7 aur v8 ke scores **compare hi nahi ho sakte.**

RAG evaluation — do types

Type	Kya evaluate hota hai
Retrieve only	Sirf retrieval — sahi documents mile?
Retrieve and generate	Poora pipeline

Do types kyun: jawaab kharab aaya — **retrieval fail hua ya generation?**

Agar retrieve-only pass ho aur final answer galat ho, toh **problem model mein hai, retriever mein nahi.** Yahi inference in do modes ka poora point hai.

Citation precision vs coverage

Precision	-> jo cite kiya, wo SAHI hai?
Coverage	-> jo cite karna chahiye tha, wo SAB cite hua?

Context matching bhi alag hai relevance se: document relevant ho sakta hai par **us situation par lagu na ho.**

Agent evaluation

Metric	Kya
Task completion rate	Kaam poora hua?
Tool usage effectiveness	Sahi tool chuna, sahi use kiya?
Reasoning quality	Multi-step mein logical progression

Agent evaluation POORA PROCESS judge karta hai, sirf jawaab nahi. Agent sahi jawaab par **galat raaste se** pahuncha — wo bhi problem hai, kyunki wo generalize nahi karega.

Deployment validation

Synthetic user workflows: poore **customer journeys** simulate karo, ek query nahi. Customer 5 sawaal poochta hai — pura flow chalta hai ya nahi?

Semantic drift: naya model "liquidity" ko usi tarah samajh raha hai? **Agar interpretation badal gayi toh sab jawaab badal jaayenge.**

Shadow deployment — sabse chalak technique

Naya model REAL production traffic par chalao
-> par uska output DISCARD kar do
-> user ko purana model hi jawaab de raha hai
-> dono ke outputs offline compare karo

	User kya dekhta hai	Risk / Cost
Shadow	Purana model	Zero risk, double cost
Canary	Naya model, thode users ko	Kam risk
Progressive	Naya, badhte users ko	Kam, ghatta hua
Blue/Green	Naya, sab ko ek saath	Zyada, par rollback turant

Exam signal: "test with production traffic without affecting users" → **shadow.**

Natural order: **shadow → canary → progressive → full rollout.**

Aakhri baat jo transcript ne kahi, aur wo important hai: evaluation system sirf quality ke liye nahi hai — wo **compliance evidence bhi hai** (Task 3.3 ka governance).

Task 5.2 — Troubleshooting

Chaar symptoms, chaar diagnostic paths

1. Incomplete responses

-> content handling
2. API failures

-> FM integration
3. Inconsistent responses->

prompt engineering
4. Slow/irrelevant

-> retrieval system

Problem 1 — Incomplete responses

Diagnosis: context window overflow. CloudWatch se token usage patterns dekho — **kaunsa document type consistently overflow karta hai.**

Sab queries problem nahi hain. Ek specific type hai. Wahi fix karo.

Fix: fixed-size se **semantic chunking** — natural boundaries (visit dates, treatment episodes) par kaato.

Truncation SILENT hoti hai. Model jo mila usi se jawaab de deta hai — **aur wo confident lagta hai.**

Isliye **alert chahiye** jab important content cut ho raha ho. Transcript mein truncation **medication lists aur treatment histories** mein ho rahi thi — sabse high-value data.

Problem 2 — API failures

Structured logging se errors categorize karo (type, severity, frequency). Transcript ka finding: **throttling errors morning rounds** ke waqt.

"API failed" se kuch pata nahi chalta — throttling hai, timeout hai, ya validation? **Sirf categorised logging batata hai.**

Request validation se error rates 40% kam huye.

Matlab: aadhe se zyada "API failures" **aapke apne malformed requests** the.

X-Ray ka finding — sabse achha example: bottleneck **AUTHENTICATION MIDDLEWARE** mein tha, model mein nahi.

Bina end-to-end tracing ke team **model optimize karti rehti aur kuch na hota.**

Load shedding: peak par kuch requests **jaan-boojh kar drop** karo, taaki baaki chalti rahein. Healthcare mein — emergency queries chalein, routine wait karein.

Problem 3 — Inconsistent responses

Yeh **context sensitivity** hai — Task 4.3 wala GenAI-specific failure mode.

Fix: specialty aur query type ke hisaab se systematic testing, version control with rollback, A/B testing jo **consistency bhi measure kare**, aur CoT analysis se reasoning failures pakadna.

Prompt complexity metrics — ulta insight:

Log sochte hain **zyada instructions = behtar jawaab.**

Aksar ulta hota hai — bahut saari conflicting ya nested instructions se model confuse ho jaata hai.

Problem 4 — Retrieval issues

Embedding visualization (SageMaker): vector space visualize karo — **clustering issues** dikhte hain.

Agar medical terminology ke embeddings theek se cluster nahi ho rahe — matlab **embedding model domain nahi samajh raha.** Yeh Task 1.5 ki "domain fit dimensions se zyada important hai" wali baat ka **diagnostic tool** hai.

Drift monitoring: medical terminology **badalti rehti hai** — new research, new terms. Purane embeddings nayi terminology nahi samajhte.

Specialized chunking: medical literature mein **research findings + methodology + conclusions saath rehne chahiye.** Alag ho gaye toh finding bina methodology ke aa jaayegi — galat interpretation.

RAG pipeline ka har step log karo: data preparation, chunking (strategy, size, **overlap, chunk counts**), ingestion, retrieval, evaluation. **Warna pata hi nahi chalega ki problem kahan hai.**

Prompt maintenance

Logs Insights se "prompt confusion" ke patterns dhoondho:

Model instruction ko SAWAAL samajh raha hai

Jawaab template ke bare mein hai, content ke bare mein nahi

Placeholder text jawaab mein aa raha hai ({{variable}})

Model conflicting instructions par comment kar raha hai

Yeh patterns logs mein dikhte hain — agar aap dhoondhein.

X-Ray prompt RENDERING pipeline par bhi: template fetch, variables bharna, context assemble karna. **Yeh model call nahi hai — aur yeh bhi slow ho sakta hai ya fail ho sakta hai.**

Version management mein performance metrics saath:

```
v6 -> accuracy 91%
v7 -> accuracy 94%
v8 -> accuracy 88%    -> rollback
```

Diagnostic map — symptom se tool tak

Symptom	Pehle dekho	Tool
Incomplete responses	Token usage vs context limit	CloudWatch metrics
API failures	Error type, severity	Structured logging
Slow responses	Kahan time ja raha hai	X-Ray tracing
Inconsistent answers	Prompt versions, variations	A/B testing, output diffing
Irrelevant retrieval	Embeddings, chunking	Embedding visualization
Wrong reasoning	Step-by-step logic	CoT analysis

Sabse zaroori seekh

Transcript ke **dono** findings dhyan dene layak hain:

1. Bottleneck AUTH middleware mein tha, model mein nahi

2. 40% "API failures" apne MALFORMED REQUESTS the

Dono baar problem wahan nahi thi jahan lag rahi thi.

Isliye end-to-end tracing aur structured logging zaroori hain — **warna aap galat cheez optimize karte rahoge.**

Domains 4 & 5 — Yaad rakhne layak dus baatein

#	Baat
1	Cost mein model selection sabse bada lever hai — prompt tuning se pehle
2	Target = sabse sasta model jo threshold meet kare , sabse achha nahi
3	Prompt caching mein static hissa shuru mein — prefix par kaam karti hai
4	Streaming perceived latency badalti hai, actual nahi
5	HNSW tez par memory zyada; ef_search query time par, m build time par
6	GenAI aise fail hota hai jahan error rate zero rehta hai — normal monitoring andhi hai
7	Golden datasets = hallucination; output diffing = inconsistency; reasoning tracing = logical error
8	Retrieve-only evaluation se pata chalta hai galti retriever ki hai ya model ki
9	Shadow = user purana model dekhta hai , canary = naya. Zero risk vs kam risk.
10	Problem aksar wahan nahi hoti jahan lagti hai — tracing aur structured logging isiliye hain